

JavaScript: Objects and Functions

Credit

○ This slide is copied and modified from Fulvio Corno, Luigi De Russis @



Outline

- Objects
- Functions
 - Closures
 - IIFE
- Dates



JavaScript: The Definitive Guide, 7th Edition Chapter 5. Objects

Mozilla Developer Network

- [Learn web development JavaScript » Dynamic client-side scripting » Introducing JavaScript objects](#)
- [Web technology for developers » JavaScript » JavaScript reference » Standard built-in objects » Object](#)
- [Web technology for developers » JavaScript » JavaScript reference » Expressions and operators » in operator](#)

JavaScript – The language of the Web

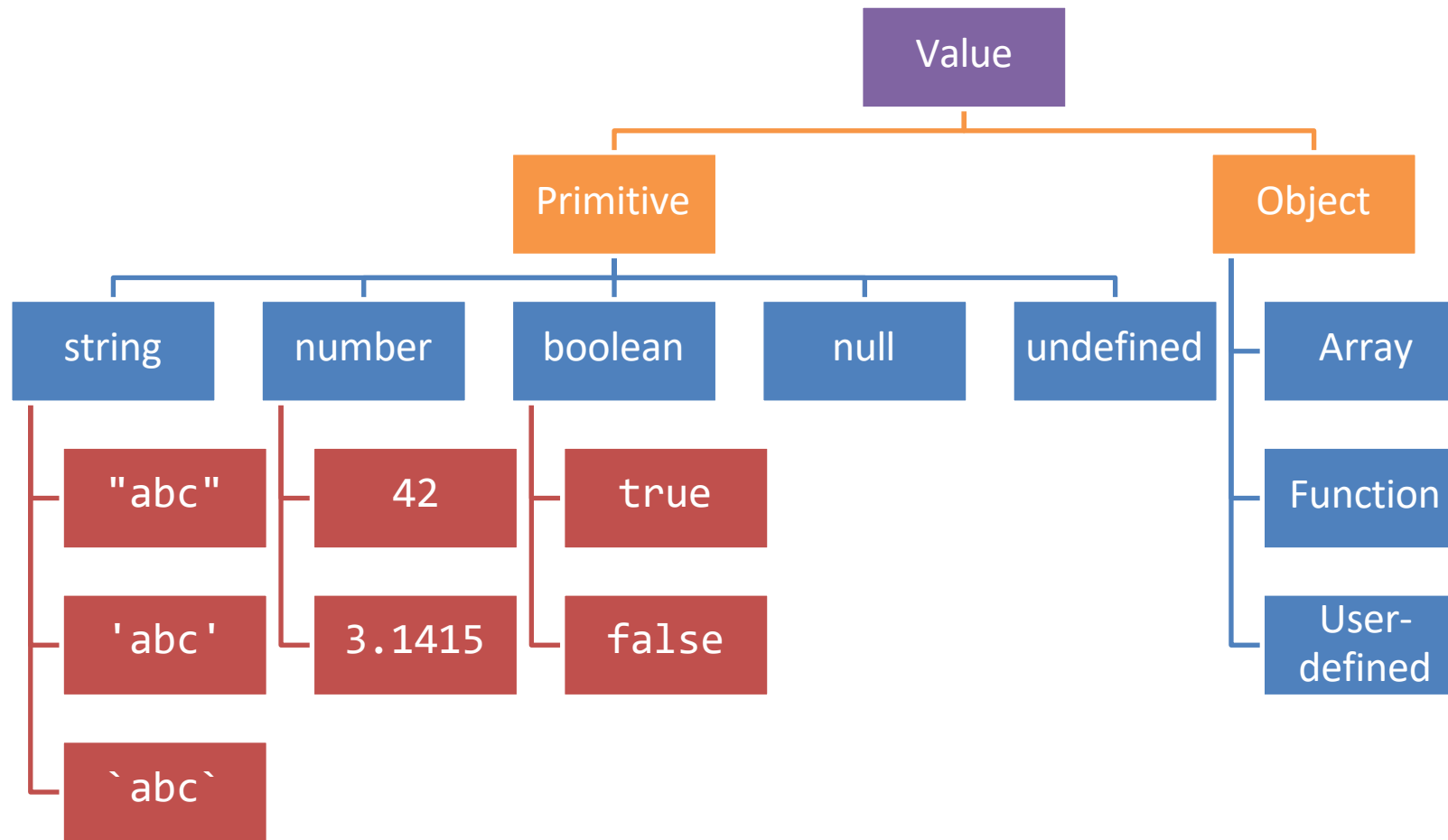
OBJECTS

Big Warnings (*a.k.a., forget Java objects*)

- In JavaScript, Objects may exist without Classes
 - Usually, Objects are **created directly**, without deriving them from a Class definition
- In JavaScript, Objects are dynamic
 - You may **add, delete, redefine** a **property** at any time
 - You may add, delete, redefine a **method** at any time
- In JavaScript, there are no access control methods
 - Every property and every method is always **public** (private/protected don't exist)
- There is no real difference between **properties and methods** (because of how JS functions work)

Values and Types

*Values have types.
Variables don't.*

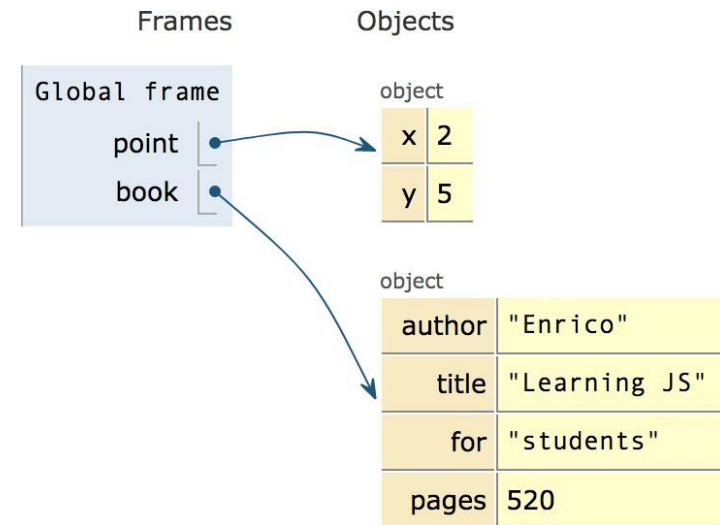


Object

- An object is an **unordered collection of properties**
 - Each property has a **name** (key), and a **value**
- You store and retrieve *property values*, through the *property names*
- Object creation and initialization:

```
let point = { x: 2, y: 5 };  
  
let book = {  
  author : "Enrico",  
  title : "Learning JS",  
  for: "students",  
  pages: 520,  
};
```

Object literals syntax:
{"name": value,
"name": value, }
or:
{name: value,
name: value, }



Object Properties

Property names are ...

- Identified as a **string**
- Must be unique in each object
- Created at object initialization
- Added after object creation
 - With assignment
- Deleted after object creation
 - With `delete` operator

Property values are ...

- Reference to any **JS value**
- Stored inside the object
- May be **primitive** types
- May be **arrays**, other **objects**, ...
 - Beware: the object stores the reference, the value is *outside*
- May also be **functions** (*methods*)

Creating Objects

- Objects can be created
 - with object literals,
 - Comma-separated list of colon-separated name:value pairs, enclosed within curly braces

```
1. let point = { x: 2, y: 5 };
```

- with the **new** keyword, and
- with the **Object.create()** function

- Almost every JavaScript object has a second object is known as a prototype
 - All objects created by object literals have the same prototype object
 - Object.prototype

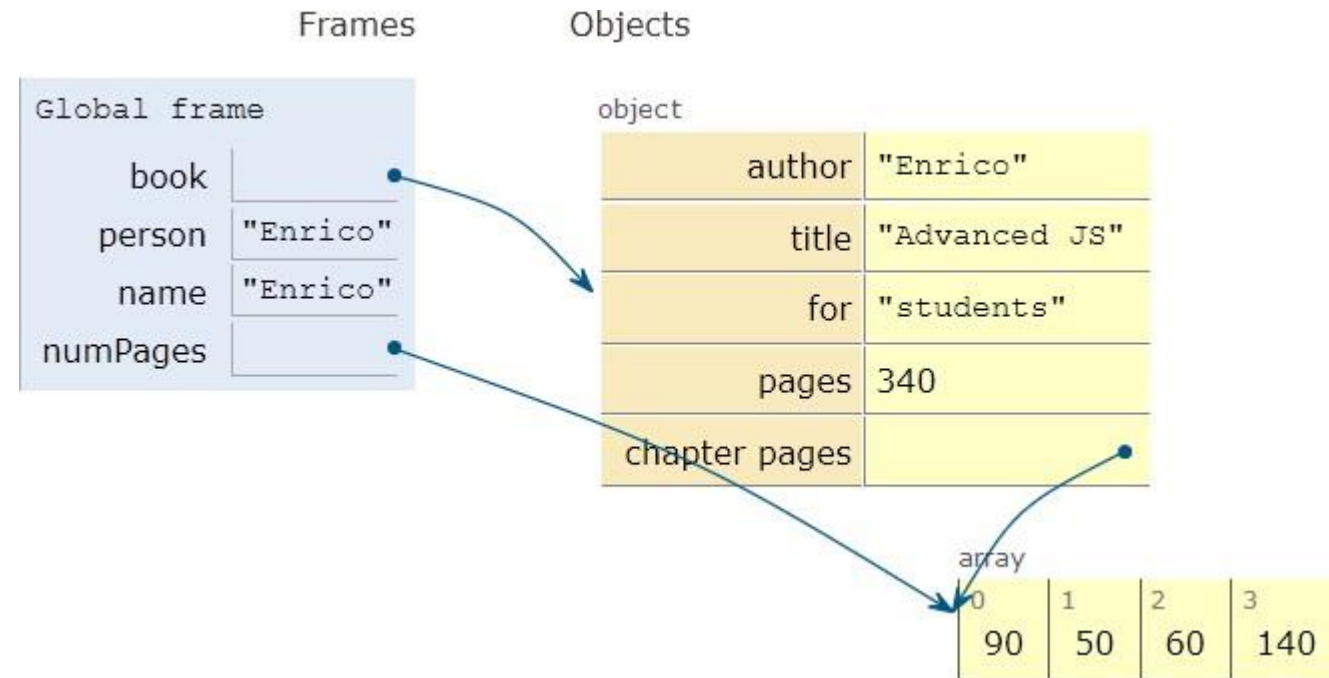
```
1. let o = new Object();
2. // Create an empty object: same as {}.
3. let a = new Array();
4. // Create an empty array: same as [].
5. let d = new Date();
6. // Create a Date object representing the current time
7. let r = new Map();
8. // Create a Map object for key/value mapping
9.
10. let o1 = Object.create({x: 1, y: 2});
11. // o1 inherits properties x and y.
12. o1.x + o1.y // => 3
```

Accessing Properties

- Dot (.) or square brackets [] notation
 - . dot notation and omitting quotes are allowed when the property name is a valid identifier, only

```
let book = {  
  author : "Enrico",  
  title : "Learning JS",  
  for: "students",  
  pages: 340,  
  "chapter pages": [90,50,60,140]  
};
```

```
let person = book.author;  
let name = book["author"];  
let numPages = book["chapter pages"];  
book.title = "Advanced JS";  
book["pages"] = 340;
```



Objects as Associative Arrays

- The `[]` syntax looks like array access, but the index is a string
 - Generally known as associative arrays
- Setting a non-existing property creates it:
 - `person["telephone"] = "0110901234";`
 - `person.telephone = "0110901234";`
- Deleting properties
 - `delete person.telephone;`
 - `delete person["telephone"];`

Computed Property Names

- Flexibility in creating object properties
 - `{[prop]:value}` -> creates an object with property name equal to the value of the variable `prop`
- `.` operator to access a property of object
 - Name of the property is expressed as an identifier
 - Identifiers must be typed literally
 - Cannot be manipulated by the program
- When you access a property of an object with the `[]` array notation
 - Can be manipulated and created
 - `address0`, `address1`, `address2`
- Beware of quotes:
 - `book["title"]` -> property called `title`
 - Equivalent to `book.title`
 - `book[title]` -> property called with the value of variable `title` (if exists)
 - If `title=="author"`, then equivalent to `book["author"]`
 - No equivalent in dot-notation

```
1. let addr = "";
2. for(let i = 0; i < 4; i++) {
3.     addr += customer[`address${i}`] + "\n";
4. }
```

Property Access Errors

- If a property is not defined, the (attempted) access returns **undefined**
 - Not an error
- It is an error to attempt to query a property of an object that does not exist
- If unsure, must check before accessing
 - Remember: **undefined** is **falsy**, you may use it in Boolean expressions
- Property access expressions will fail if the lefthand side of the `.` is **null** or **undefined**

```
1. let o = {x: 1};
2. let p = null;
3. o && o.x // => o.x
4. p && p.x // => null
```

```
1. let o = {};
2. book.subtitle // => undefined:
3. let len = book.subtitle.length; // Not have length
4.
5. let surname = undefined; // 1st
6. if (book) {
7.     if (book.author) {
8.         surname = book.author.surname;
9.     }
10. }
11. // 2nd
12. surname = book && book.author && book.author.surname;
```

- If both operands are truthy, the operator returns a truthy value (NOT boolean value)
 - Return the value on the right
- Otherwise, one or both operands must be falsy, and the operator returns a falsy value
 - Return the value on the left

Iterating over Properties

- `for .. in` iterates over the properties

```
for( let a in {x: 0, y:3}) {  
    console.log(a) ;  
}
```

```
x  
y
```

```
let book = {  
    author : "Enrico",  
    pages: 340,  
    chapterPages: [90,50,60,140],  
};  
  
for (const prop in book)  
    console.log(`${prop} = ${book[prop]}`);
```

```
author = Enrico  
pages = 340  
chapterPages = 90,50,60,140
```

Iterating over Properties

- All the (enumerable) properties names (keys) of an object can be accessed as an array, with:

- `let keys = Object.keys(book) ;`

```
[ 'author', 'pages' ]
```

- All pairs [key, value] are returned as an array with:

- `let keys_values = Object.entries(book)`

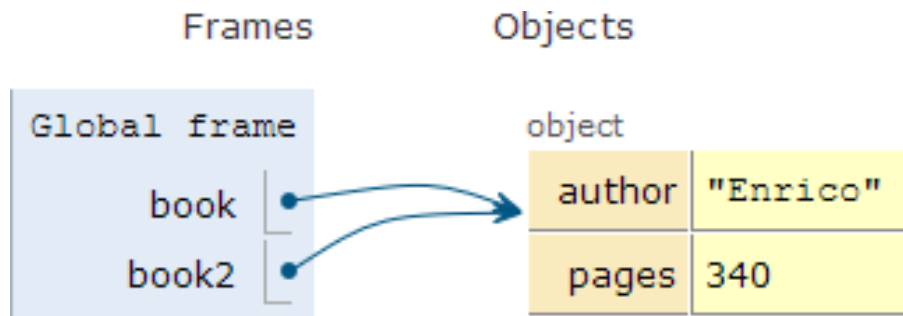
```
[ [ 'author', 'Enrico' ], [ 'pages', 340 ] ]
```

```
1. let book = {  
2.   author : "Enrico",  
3.   pages: 340,  
4.   chapterPages: [90,50,60,140],  
5. };
```

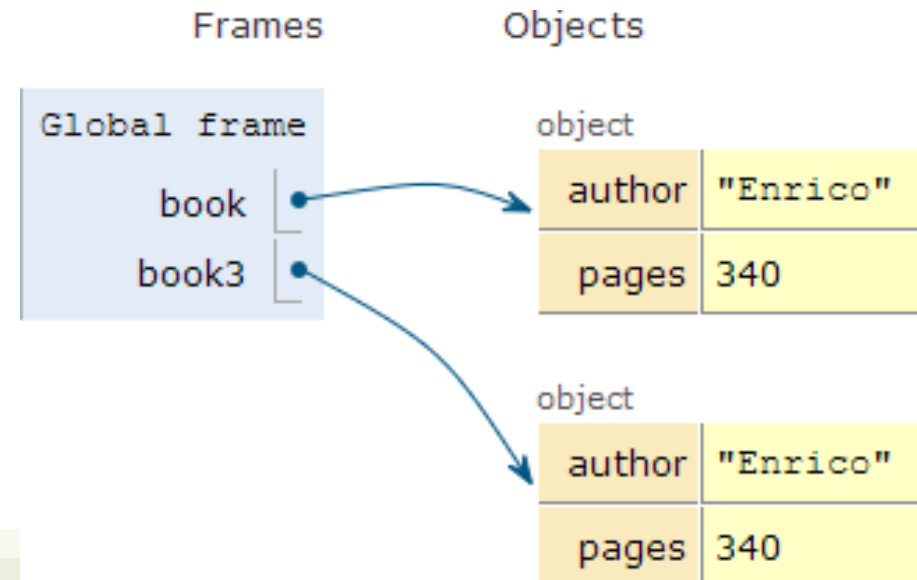
```
> Object.keys(book) ;  
< ▶ (3) ['author', 'pages', 'chapterPages']  
> Object.entries(book)  
< ▼ (3) [Array(2), Array(2), Array(2)] ⓘ  
  ▶ 0: (2) ['author', 'Enrico']  
  ▶ 1: (2) ['pages', 340]  
  ▶ 2: (2) ['chapterPages', Array(4)]  
    length: 3  
  ▶ [[Prototype]]: Array(0)
```

Copying Objects

```
let book = {  
  author : "Enrico",  
  pages: 340,  
};  
  
let book2 = book;  // ALIAS
```



```
let book = {  
  author : "Enrico",  
  pages: 340,  
};  
  
let book3 =                               // COPY  
  Object.assign({}, book);
```



Object.assign

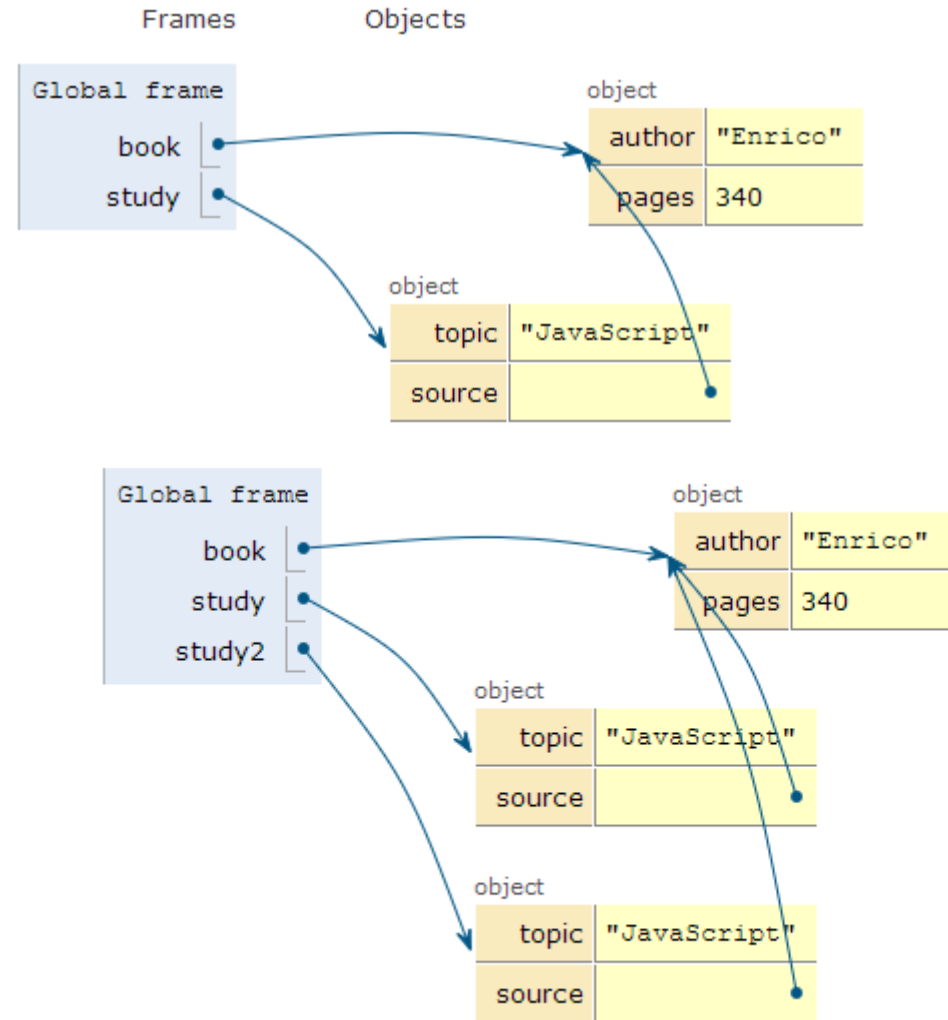
- `let new_object = Object.assign(target, source);`
- Assigns all the properties from the source object to the target one
- The target may be an existing object
- The target may be a new object: `{}`
- Returns the target object (after modification)

Beware! Shallow copy, only

```
let book = {  
  author : "Enrico",  
  pages: 340,  
};
```

```
let study = {  
  topic: "JavaScript",  
  source: book,  
};
```

```
let study2 = Object.assign({}, study);
```



Kind of Object Copies

- 3 kinds of object copies
- Reference copy
 - Assignment operator '='
 - Not a true copy
 - Both variables point to same object
 - Changes made to the original object will be reflected in the copied object, and vice versa

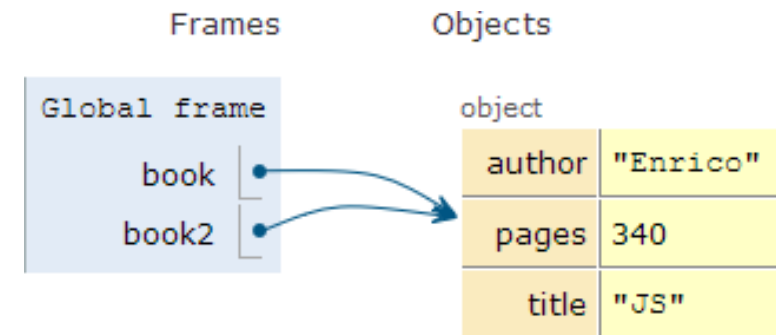
```
1. const obj1 = { a: 1, b: 2};  
2. const obj2 = obj1;  
3. console.log( obj1 === obj2 ); // true
```

```
1. const obj1 = { a:1, b:2 };  
2. const obj2 = obj1;  
  
3. obj2.a = 100;  
  
4. console.log( obj1.a ); // 100
```

Merge properties (on existing object)

- `Object.assign(target, source, default values, ..);`

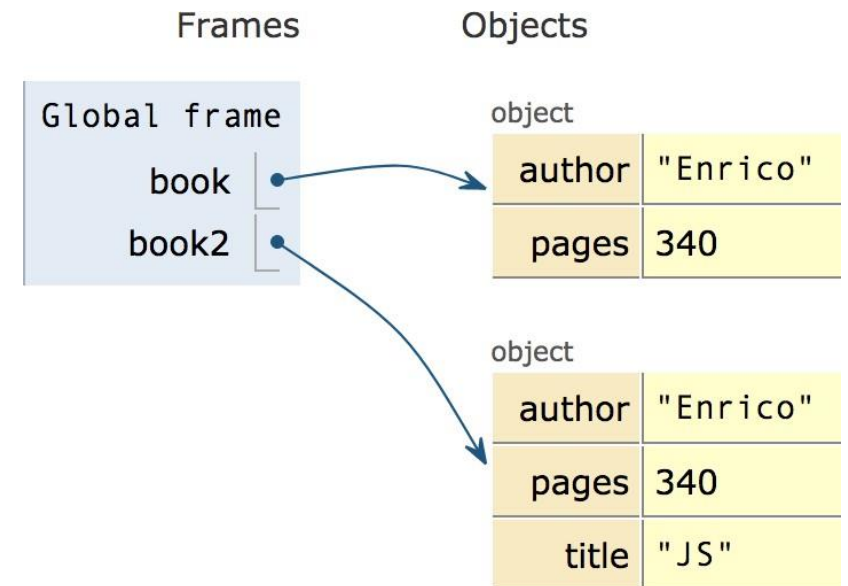
```
let book = {  
  author : "Enrico",  
  pages: 340,  
};  
  
let book2 = Object.assign( book, {title: "JS"} );
```



Merge properties (on new object)

- `Object.assign(target, source, default values, ..);`

```
let book = {  
  author : "Enrico", pages: 340,  
};  
  
let book2 = Object.assign({}, book, {title: "JS"} );
```



Copying with Spread Operator (ES9 – ES2018)

```
let book = {  
  author : "Enrico",  
  pages: 340,  
};  
  
let book2 = {...book, title: "JS"};  
let book3 = { ...book2 } ;  
console.log(book2);
```

```
{ author: 'Enrico', pages: 340, title: 'JS' }
```

```
const {a,b,...others} =  
  {a:1, b:2, c:3, d:4};  
  
console.log(a);  
console.log(b);  
console.log(others);
```

```
1  
2  
{ c: 3, d: 4 }
```

Shallow and Deep Copy

○ Shallow copy

- Creating a new object with its properties referencing the same values or objects

```
1 const userObj = { name: 'Billy', age: '22', address: { country: 'VietNam' } }
2 const cloneUserObj = { ...userObj }
3
4 cloneUserObj.address.country = 'No'
5
6 console.log(userObj)
7 console.log(cloneUserObj)
8 // { name: 'Billy', age: '22', address: { country: 'no' } }
9 // { name: 'Billy', age: '22', address: { country: 'no' } }
10
```

```
1 const userObj = { name: 'Billy', age: '22', address: { country: 'VietNam' } }
2 const cloneUserObj = Object.assign(userObj)
3
4 cloneUserObj.address.country = 'No'
5
6 console.log(userObj)
7 console.log(cloneUserObj)
8 // { name: 'Billy', age: '22', address: { country: 'no' } }
9 // { name: 'Billy', age: '22', address: { country: 'no' } }
```

○ Deep copy

- Creating a new object is an exact copy of an existing object
- This includes copying all its properties and any nested objects, instead of references

```
1 const userObj = { name: 'Billy', age: '22', address: { country: 'VietNam' } }
2 const cloneUserObj = JSON.parse(JSON.stringify(userObj))
3
4 cloneUserObj.address.country = 'No'
5
6 console.log(userObj)
7 console.log(cloneUserObj)
8 // { name: 'Billy', age: '22', address: { country: 'VietNam' } }
9 // { name: 'Billy', age: '22', address: { country: 'no' } }
```

Checking if Properties Exist

- Operator **in**

- Returns true if property is in the object

```
let book = {  
  author : "Enrico",  
  pages: 340,  
};  
  
console.log('author' in book);  
delete book.author;  
console.log('author' in book);
```

```
true  
false
```

- Do NOT use with Array

```
const v=['a','b','c'];  
  
console.log('b' in v);  
  
console.log('PI' in Math);
```

```
false  
true
```

Object Creation (equivalent methods)

- By object literal

- `const point = {x:2, y:5} ;`

- By object literal (empty object)

- `const point = {} ;`

- By constructor:

- `const point = new Object() ;`

- By object static method create:

- `const point = Object.create({x:2,y:5});`





JavaScript – The language of the Web

FUNCTIONS

Functions

- **One of the most important** elements in JavaScript
- Delimits a block of code with a private scope
- Can accept parameters and returns one value
 - Can also be an object
- Functions themselves **are objects** in JavaScript
 - They can be **assigned** to a variable
 - Can be **passed** as an argument
 - Used as a **return** value

Declaring functions: 3 ways

1) Classic

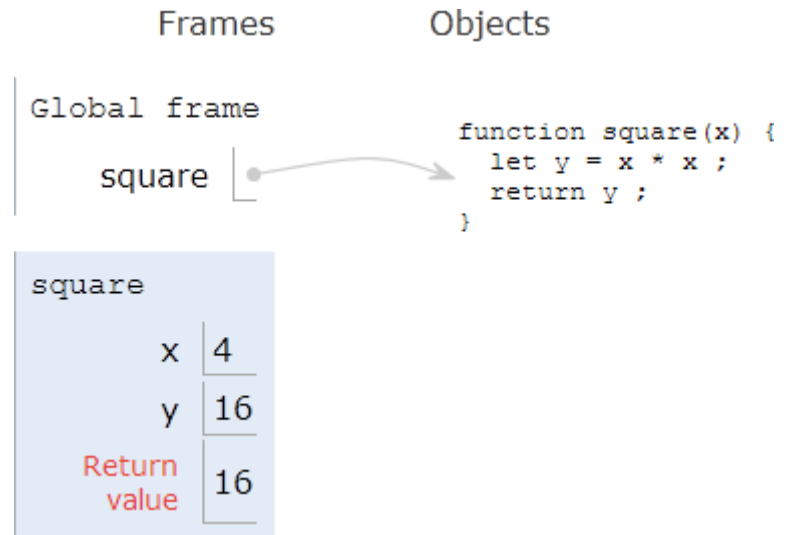
```
function do(params) {  
  /* do something */  
}
```

Classic Functions

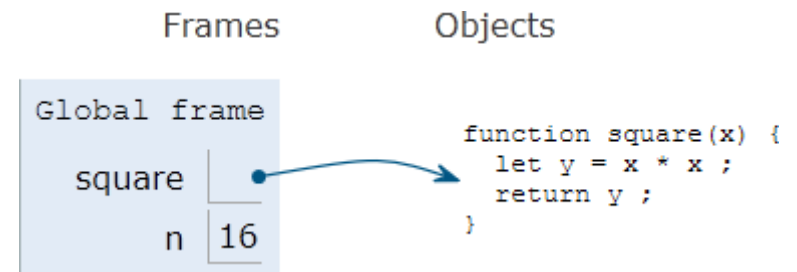
```
function square(x) {  
  let y = x * x ;  
  return y ;  
}
```

```
let n = square(4) ;
```

During
execution



After
execution



Parameters

- Comma-separated list of parameter names
 - May assign a default value, e.g., `function(a, b=1) {}`
- Parameters are passed by-value
 - Copies of the reference to the object
- When a function is invoked with fewer arguments than declared parameter, additional parameters are set to their default value
 - Normally 'undefined'
- Check missing/optional parameters with:
 - `if(p===undefined) p = default_value ;` instead `p = p || default_value ;`
 - `||` operator returns its first argument if it is truthy and otherwise returns its second argument

Variable Number of Parameters

- Syntax for functions with variable number of parameters, using the `...` operator
 - Called “rest”
 - `function fun (par1, par2, ...arr) { }`
- The “rest” parameter must be the last, and will deposit all extra arguments into an array

```
1. function sumAll(initVal, ...arr) {  
2.     let sum = initVal;  
3.     for (let a of arr)  
4.         sum += a;  
5.     return sum;  
6. }  
7. sumAll(0, 2, 4, 5); // 11
```

Declaring Functions: 3 ways

1) Classic

```
function do(params) {  
  /* do something */  
}
```

2a) Function expression

```
const fn = function(params) {  
  /* do something */  
}
```

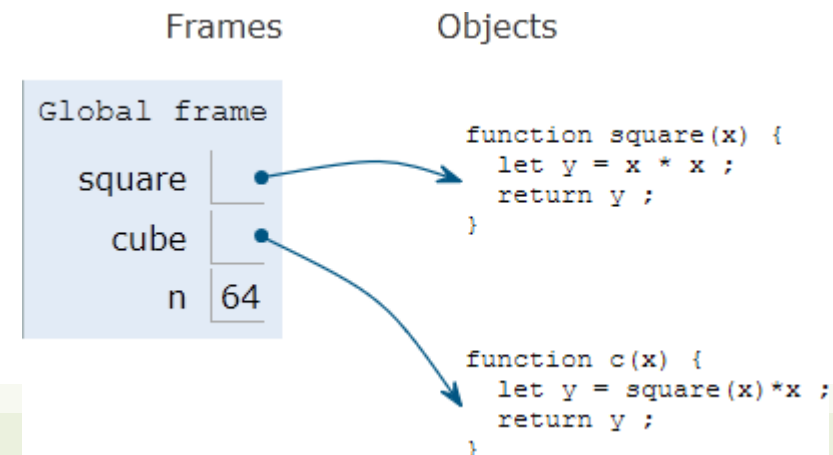
2b) Named function expression

```
const fn = function do(params) {  
  /* do something */  
} // for recursive
```

Function Expression: Indistinguishable

- The expression `function(){}` creates a new object of type 'function' and returns the result `n = cube(4)`;
- Any variable may "refer" to the function and call it
- You can also store that reference into an array, an object property, pass it as a parameter to a function, redefine it, ...
 - So called method and callback

```
1. function square(x) {  
2.     let y = x * x ;  
3.     return y ;  
4. }  
5. let cube = function c(x) {  
6.     let y = square(x)*x ;  
7.     return y ;  
8. }  
9. let n = cube( 4 );
```



Declaring functions: 3 ways

1) Classic

```
function do(params) {  
  /* do something */  
}
```

2a) Function expression

```
const fn = function(params) {  
  /* do something */  
}
```

3) Arrow function

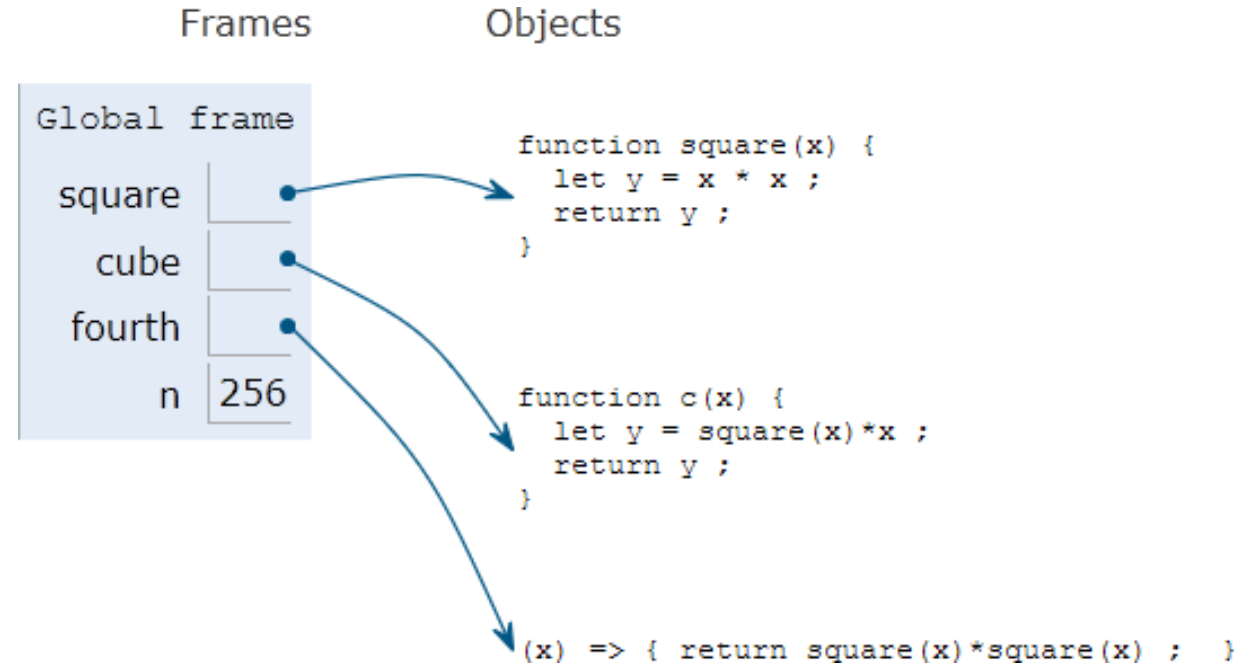
```
const fn = (params) => {  
  /* do something */  
}
```

2b) Named function expression

```
const fn = function do(params) {  
  /* do something */  
}
```

Arrow Function: just a shortcut

```
1. function square(x) {  
2.     let y = x * x ;  
3.     return y ;  
4. }  
5.  
6. let cube = function c(x) {  
7.     let y = square(x)*x ;  
8.     return y ;  
9. }  
10. let fourth = (x) => {  
11.     return square(x)*square(x);  
12. }  
13.  
14. let n = fourth(4) ;
```



Parameters in Arrow Functions

- `const fun = () => { /* do something */ } // zero parameter`
- `const fun = param => { /* do */ } // 1 parameter, can omit ( )`
- `const fun = (param) => { /* do something */ }`
- `const fun = (par1, par2) => { /* smtg */ } // 2 params`
- `const fun = (par1 = 1, par2 = 'abc') => { /* */ } // default`
- Concise syntax of arrow functions makes them ideal when you need to pass one function to another function

```
1. // Make a copy of an array with null elements removed.  
2. let filtered = [1,null,2,3].filter(x => x !== null); //filtered == [1,2,3]  
3. // Square some numbers:  
4. let squares = [1,2,3,4].map(x => x*x); //squares == [1,4,9,16]
```

Return Value

- Default: **undefined**
 - Use **return** to return a value
 - Only one value can be returned
 - However, objects (or arrays) can be returned
- ```
const fun = () => { return ['hello', 5] ; }
const [str, num] = fun() ;
console.log(str) ;
```
- Arrow functions have **implicit return** if there is only one value

```
let fourth = (x) => { return square(x)*square(x) ; }
let fourth = x => square(x)*square(x) ;
```

# Nested Functions

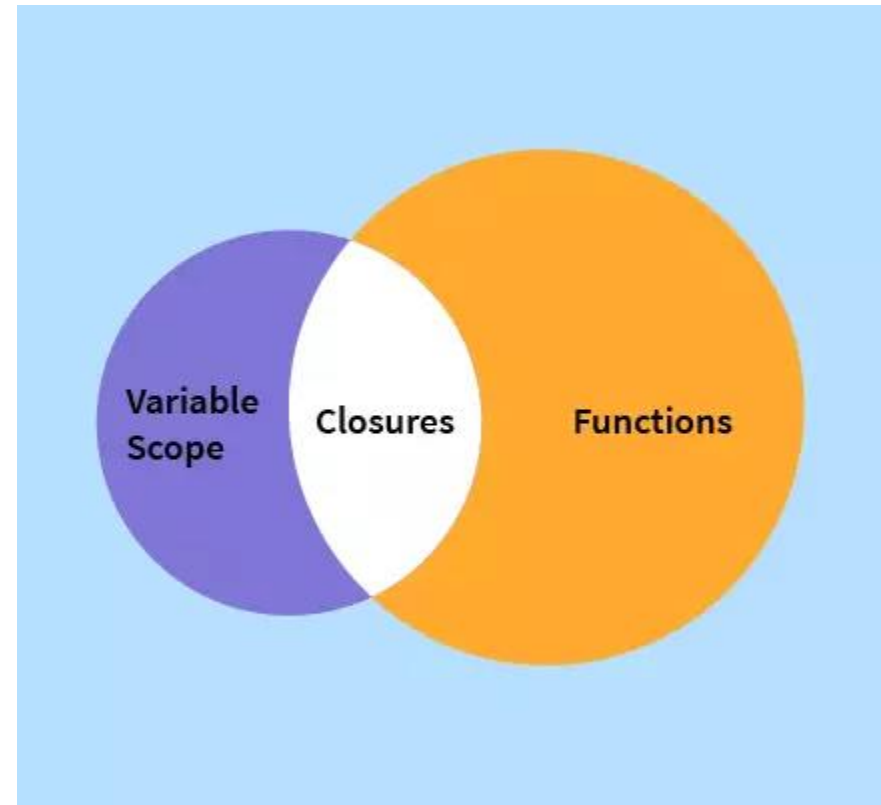
- Function can be nested, i.e., defined within another function

```
1. function hypotenuse(a, b) {
2. const square = x => x*x ;
3. return Math.sqrt(square(a) + square(b));
4. }
5.
6. function hypotenuse(a, b) {
7. function square(x) { return x*x; }
8. return Math.sqrt(square(a) + square(b));
9. }
```

- The inner function is scoped within the external function and cannot be called outside
- The inner function might access variables declared in the outside function

# Closure: definition (somewhat cryptic)

- Really: one of the most important concepts in JS
- A closure is a name by which a nested function executed after the execution of the outer function can still access outer function's scope



<https://medium.com/@vvkchandra/learn-javascript-closures-through-the-laws-of-karma-49d32d35b3f7>

# Closures

- JS uses lexical scoping
  - Each new functions defines a scope for the variables declared inside
  - Nested functions may access the scope of all enclosing functions
- Every function object remembers the scope where it is defined, even after the external function is no longer active
  - $\Rightarrow$  Closure

```
1. "use strict" ;
2. function greeter(name) {
3. const myname = name ;
4. const hello = function () {
5. return "Hello " + myname ;
6. }
7. return hello ; // NOT hello() ;
8. }
9.
10. const helloTom = greeter("Tom") ;
11. const helloJerry = greeter("Jerry") ;
12. console.log(helloTom()) ;
13. console.log(helloJerry()) ;
```

# Closures

- **hello** accesses the variable **myname**, defined in the outer scope
- The function is returned (as **helloTom** or **helloJerry**)
- Each of the functions “remembers” the reference to **myname**, when it was defined
- The variable **myname** goes out of scope, but is not destroyed
  - Still accessible (referred) by **hello** functions

```
1. "use strict" ;
2. function greeter(name) {
3. const myname = name ;
4. const hello = function () {
5. return "Hello " + myname ;
6. }
7. return hello ;
8. }
9.
10. const helloTom = greeter("Tom") ;
11. const helloJerry = greeter("Jerry") ;
12. console.log(helloTom()) ;
13. console.log(helloJerry()) ;
```

greeter

hello scope

```
1. let scope = "global scope";
2. // A global variable
3. function checkscope() {
4. let scope = "local scope";
5. // A local variable
6. function f() { return scope; }
7. // Return the value in scope here
8. return f();
9. }
10. checkscope() // => "local scope"
```

- checkscope( ) now just returns its result

```
1. let scope = "global scope";
2. // A global variable
3. function checkscope() {
4. let scope = "local scope";
5. // A local variable
6. function f() { return scope; }
7. // Return the value in scope here
8. return f;
9. }
10. checkscope()() // =>
```

- checkscope( ) now just returns the nested function object itself
  - where **scope** is not in the global scope

# Closures to Emulate Objects (Private Variable)

```
"use strict" ;

function counter() {
 let value = 0 ;

 const getNext = () => {
 value++;
 return value;
 }

 return getNext ;
}
```

```
const count1 = counter() ;
console.log(count1()) ;
console.log(count1()) ;
console.log(count1()) ;
```

```
const count2 = counter() ;
console.log(count2()) ;
console.log(count2()) ;
console.log(count2()) ;
```

```
1
2
3
1
2
3
```

# Using Closures to Emulate Objects (with methods)

```
1. function counter() {
2. let n = 0;
3. // return an object,
4. // containing two function-valued
5. // properties
6. return {
7. count: function() { return n++; },
8. reset: function() { n = 0; }
9. };
10. }
```

```
1. let c = counter(), d = counter();
2. // Create two counters
3. c.count() // => 0
4. d.count() // => 0: they count independently
5. c.reset(); // reset() and count() methods share state
6. c.count() // => 0: because we reset c
7. d.count() // => 1: d was not reset
```

# Function Declaration vs Function Expression

- To declare a function, you use the function keyword and specify a name for function
  - "declared" a function called `generateIntro`
  - Use the function keyword followed by the name of the function: "`generateIntro`"

```
1. function generateIntro(name) {
2. return `Hi, my name is ${name}`
3. }
4. const dillion = generateIntro("Dillion");
5. console.log(dillion)
```

- Create a function expression and assign it to a variable that can be called. 2 ways
  - Use the function keyword without a name, which makes it an anonymous function

```
1. const generateIntro = function(name) {
2. return `Hi, my name is ${name}`
3. }
4. const dillion = generateIntro("Dillion")
5. console.log(dillion)
6. const generateIntro = (name) => {
7. return `Hi, my name is ${name}`
8. }
9. const dillion = generateIntro("Dillion")
10. console.log(dillion)
```

- Arrow function expressions
  - They cannot be declared; they can only be expressed

# Differences Between These

- Expressed functions cannot be used before initialization

```
1. const result = sum(20, 50)
2. console.log(result)
3. function sum(num1, num2) {
4. return num1 + num2
5. } // declaration NO ERROR
6.
7. const result = sum(20, 50)
8. console.log(result)
9. const sum = function(num1, num2) {
10. return num1 + num2
11. } // expression error
```

- When you declare variables with **let** or **const** (like we did for **sum** here), they are hoisted, but without a default initialization

- Expressed functions need to be assigned to be used later

- In declaration, you already have the name: function **name**....
- So you can use the function later: **name()**.

```
1. const printName = function(firstname, lastname) {
2. console.log(`${firstname} ${lastname}`)
3. }
```

- We **assigned** the function expression to **printName**
- Now we can use that function logic later by calling **printName()**
- You can execute the function instantly In such cases, you do not need a name
- Use expression instead of declaration

# Function Expression VS IIFE

- Function Expression is stored in a variable

```
1. const eatFruit = function() {
2. console.log('a bite of the apple');
3. }
4.
5. eatFruit(); // a bite of the apple
```

- Call eatFruit() to invoke the function
  - The parenthesis **()** at the end of the function name is what is invoking it

- Immediately Invoked Function Expressions
- IIFEs are immediately invoked after creation
  - Itself doesn't need to invoke explicitly

```
1. (function() {
2. console.log('deeecode')
3. })()
4.
5. (() => {
6. console.log('deeecode')
7. })()
```

# Immediately Invoked Function Expressions

- Syntax

- Wrap parentheses around functions
  - This creates the function expression
- Add a second pair of parentheses at the end to immediately invoke it

```
(function(a,b){
 return a + b;
})(2, 3) ; // 5
```

- Once the function is defined, it is immediately invoked

```
let num = (function() {
 let a = 3 ;
 return a ;
}) () ;
```

<https://flaviocopes.com/javascript-iife/>  
<https://medium.com/@vvkchandra/essential-javascript-mastering-immediately-invoked-function-expressions-67791338ddc6>

# Why Use IIFE?

- Creating a local (private) scope
  - Any variables declared inside it are not accessible outside the function
- Avoiding naming conflicts
  - Can create a separate namespace for variables to avoid conflicts between function names and variable names
- Useful for one-time setup tasks or initializing variables before the main program runs.

```
1. // Global scope
2. var globalVariable = "global ";
3. (function () { // Local scope inside IIFE
4. var globalVariable = "local inside ";
5. console.log(globalVariable); // local inside
6. })();
7. console.log(globalVariable); // global
```

# Using IIFE to Emulate Objects (with methods)

- Can use IIFE to emulate object's property
  - That is a method

```
1. "use strict"
2. const c = (function() {
3. let n = 0;
4. return {
5. count: function() {
6. return n++;
7. reset: function() {
8. n = 0;
9. };
10. })();
```

```
console.log(c.count());
console.log(c.count());
c.reset();
console.log(c.count());
console.log(c.count());
```

```
0
1
0
1
```

# First-class Functions VS Higher-order Functions

- Functions in that language are treated as values like any other variable
  - Assigning function to variables
    - **greet** function is defined as an anonymous function and assigned to the variable **greet**
  - Passing functions as arguments
    - **calculate** function takes another function (**operation**) as an argument
  - Returning function from function
    - **multiplier** function returns another function that can be used to calculate the product of a number and a factor

```
1. // assigning functions to variables
2. const greet = function(name) {
3. console.log(`Hello, ${name}!`);
4. }
5. greet('Alice'); // Hello, Alice
6.
7. // passing functions as arguments
8. function calculate(operation, num1, num2) {
9. return operation(num1, num2);
10.}
11.function add(a, b) {
12. return a + b;
13.}
14.console.log(calculate(add, 5, 3);
15.// returning functions from functions
16.function multiplier(factor) {
17. return function(x) {
18. return x * factor;
19.}
20.const double = multiplier(2);
21.console.log(double(5)); // 2 X 5 = 10
```

# First-class Functions VS Higher-order Functions

- Higher-order functions are functions that work on other functions
  - Taking one or more functions as arguments
    - Function that is passed as an argument is called a **callback** function
  - Returning function as its result
- It operates on other functions, either by using them to perform tasks or by producing new functions
  - Powerful concept in JavaScript and functional programming
  - Enabling code that is more concise, modular, and expressive.

```
1. function callbackFunction() {
2. console.log('I am a callback function');
3. }
4.
5. // higher order function
6. function higherOrderFunction(func) {
7. console.log('I am higher order function')
8. func()
9. }
10. higherOrderFunction(callbackFunction);
```

- The **map** and **filter** function is a higher-order function that takes an array and a callback function as arguments

# Real-World Scenarios

- Event handling
  - Higher-order functions allow you to create event listeners that respond to various events by passing functions as callbacks
- Array manipulation
  - Working with arrays often involves complex transformations and filtering
  - Such as **map**, **filter**, and **reduce**
- Asynchronous operations
  - **setTimeout** and **fetch** use callbacks or return promises, allowing you to manage asynchronous behavior seamlessly

```
1. button.addEventListener('click', function() {
2. // Handle button click event
3. });
4. // array manipulation
5. const number = [1,2,3,4,5];
6. const doubled = number.map(function (num) {
7. return num * 2;
8. });
9.
10. setTimeout(function() {
11. // code to run after a delay
12. }, 1000);
```